

RUST PROGRAMMING

Modules and Crates

Kamal kumar mukiri, Apt Computing Labs

29/05/26

Apt Computing Academy

MODULES AND CRATES

Modules

- Introduction
- Defining and accessing
- Visibility and Privacy
- Organizing

Crates

- Importing Third Party Crates
- Examples

Introduction

Crates: In Rust, a "crate" is the fundamental unit of compilation, packaging, and distribution.

- Self-contained
- Focused-Responsibility
- Encapsulation
- Reusable
- Development Management
- Cohesion

Modules

- Introduction

- Defining and accessing
- Visibility and Privacy
- Organizing

Crates

- Importing Third Party Crates
- Examples

A sample example:

1. How to create a crate (library) ?
2. How to include library as dependency in my application ?
3. How to invoke the APIs from Library ?

Modules

- Introduction

- **Defining and accessing**

- Visibility and Privacy

- Organizing

Crates

- Importing Third Party Crates

- Examples

Types of Modules

- **Inline Modules:** Define the module content directly in the same file.
- **File-based Modules:** Define the module in a separate file to keep the code more organized.

Modules

- Introduction

- Defining and accessing

- Visibility and Privacy

- Organizing

Crates

- Importing Third Party Crates

- Examples

In-line Modules: Example

```
1 1 // Defining module
2  mod math {
3      pub fn add(a:i32, b:i32) -> i32 {
4          return a+b;
5      }
6  3 // Defining sub-module
7      pub mod trigonometric {
8          pub fn trig_fun1() -> Option<String> {
9              return Some(String::from("returning from trig_fun1"));
10         }
11     }
12 }
13
14 fn main() {
15     // Calling function from module
16     2 println!("Result = {}", math::add(1,2));
17
18     //Calling function from sub module of "math"
19     4 println!("trig_fun1 output = {:?}", math::trigonometric::trig_fun1());
20 }
```

```
• > ./inline_module
Result = 3
trig_fun1 output = Some("returning from trig_fun1")
```

1. Module should be defined with start “mod” and module name.
2. Invoking API ”add” from module “math”.
3. Defining sub-module
4. Invoking API from sub-module

Keyword “mod”:

What is mod Keyword ?

- The mod keyword is used to **declare a module**.
- It defines the module structure in your codebase, making it possible to include files or internal modules in your project.
- The mod declaration tells the compiler where to look for the module's code.

When to use mod?

- You use mod when you want to **define** a module inside your project.
- You can create a new module inside the same file or in a separate file.

Modules

- Introduction

- Defining and accessing

- Visibility and Privacy

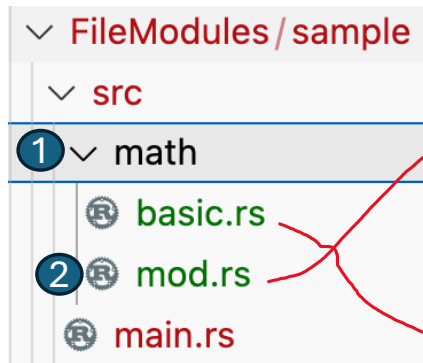
- Organizing

Crates

- Importing Third Party Crates

- Examples

File based Modules: Example



```
1 pub mod basic;
```

```
1 // math.rs
2
3 pub fn add(a: i32, b: i32) -> i32 {
4     a + b
5 }
6
7 pub fn subtract(a: i32, b: i32) -> i32 {
8     a - b
9 }
```

```
1 mod math;
2 fn main() {
3     println!("Hello, world!");
4     //calling function "add" from module "math"
5     println!("Result = {}", math::basic::add(1,2));
6 }
```

1. "math" is not a simple module, it has many functionalities
2. The `mod.rs` file serves as the entry point for the "math" module, allowing you to group multiple modules inside "math".

Prelude:

- In Rust, the **prelude** is a set of commonly used items (functions, traits, macros, etc.) that are automatically imported into every Rust program without needing to explicitly specify them. It's designed to make common tasks easier by reducing the need to import frequently used types and functions manually.

Modules

- Introduction

- Defining and accessing

- Visibility and Privacy

- Organizing

Crates

- Importing Third Party Crates

- Examples

Prelude:

Steps for Creating a Custom Prelude:

- 1. Create a prelude module:** Define a separate module in your crate (often named prelude) where you re-export the items (traits, structs, functions) you want to make easily available to the users.
- 2. Re-export items in the prelude module:** In the prelude module, re-export the most commonly used items from your crate.
- 3. Users can use the prelude:** Users of your crate can then simply import your prelude, making all of its contents accessible without needing to explicitly list them out.

Modules

- Introduction

- Defining and accessing

- Visibility and Privacy

- Organizing

Crates

- Importing Third Party Crates

- Examples

Prelude: Example

```
// src/lib.rs
pub mod my_module;
pub mod prelude; // Define a `prelude` module
```

1

```
// src/my_module.rs
pub struct MyStruct;

pub fn my_function() {
    println!("Hello from my_function!");
}

pub trait MyTrait {
    fn do_something(&self);
}

impl MyTrait for MyStruct {
    fn do_something(&self) {
        println!("MyStruct does something!");
    }
}
```

2

```
// src/prelude.rs
// Re-export commonly used items here
pub use crate::my_module::{MyStruct, MyTrait, my_function};
```

3

```
// Main file or another crate
use my_crate::prelude::*;

fn main() {
    let instance = MyStruct;
    instance.do_something(); // Using `MyTrait`

    my_function(); // Using the re-exported function
}
```

4

Modules

- Introduction
- Defining and accessing
- Visibility and Privacy
- Organizing

Crates

- Importing Third Party Crates
- Examples

Creating lib and application in same project:

- Create a New Cargo Project

```
cargo new my_project --lib
```

- Add the Binary Entry Point (main.rs)
- Write the Code for the Library (lib.rs)
- Write the Code for the Binary (main.rs)

Modules

- Introduction

- Defining and accessing

- Visibility and Privacy

- Organizing

Crates

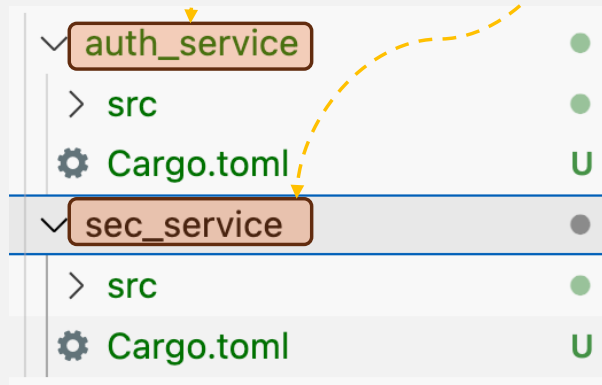
- Importing Third Party Crates

- Examples

Lib, app as separate projects:

Step 1: Create the **Library** and **Application**

```
➤ cargo new --lib auth_service  
➤ cargo new --bin sec_service
```



Cont...

Step 2: Include the Library in Application

- Add details about library and its path in **Cargo.toml** (**Application environment**)

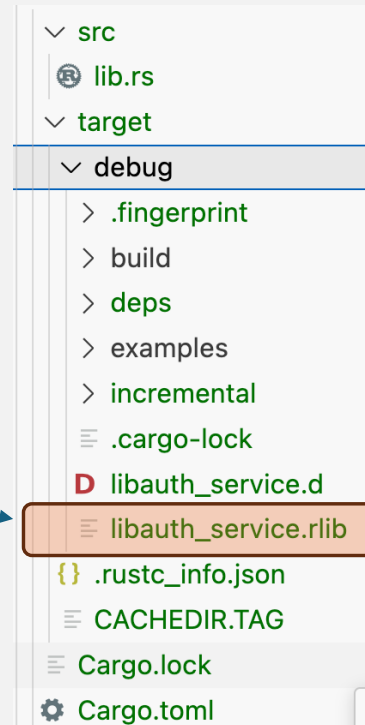
```
1  [package]
2  name = "sec_service"
3  version = "0.1.0"
4  edition = "2021"
5
6  [dependencies]
7  auth_service = {path = "../auth_service"}
8  |
```

Cont...

Step 3: Add functionality to library, compile and find .rlib

```
1  #[derive(Debug)]
   0 implementations
2  pub enum AuthError {
3      AuthSuccess,
4      AuthError
5  }
6
7  pub fn verify_authentication(user:&str, pass:&str) -> Result<AuthError, String> {
8      println!("Going to to authenticate user: {} and pass: {}", user, pass);
9      return Ok(AuthError::AuthSuccess);
10 }
11
```

➤ cargo build



Cont...

Step 4: Include library and invoke API

```
1 use auth_service;
2 fn main() {
3     println!("Hello, world!");
4     let result = auth_service::verify_authentication("Kamal", "@123");
5     println!("Result = {:?}", result);
6 }
```

➤ cargo build

```
> cargo build --verbose
Compiling auth_service v0.1.0 (/Users/kamalmukiri/Documents/GitHub/Courses/APTRUSTSESS01/Classes/Kamal/Sep30_Modules_IntegerOverflow/Modules/auth_service)
running rustc --crate-name auth_service --edition=2021 /Users/kamalmukiri/Documents/GitHub/Courses/APTRUSTSESS01/Classes/Kamal/Sep30_Modules_IntegerOverflow/Modules/auth_service/src/lib.rs --error-format=json --json=diagnostic-rendered-ansi,artifacts,future-incompat --diagnostic-width=248 --crate-type lib --emit=dep-info,metadata,link -C embed-bitcode=no -C debuginfo=2 -C split-debuginfo=unpacked -C metadata=e04437df63bd6c9b -C extra-filename=e04437df63bd6c9b --out-dir /Users/kamalmukiri/Documents/GitHub/Courses/APTRUSTSESS01/Classes/Kamal/Sep30_Modules_IntegerOverflow/Modules/sec_service/target/debug/deps --incremental=/Users/kamalmukiri/Documents/GitHub/Courses/APTRUSTSESS01/Classes/Kamal/Sep30_Modules_IntegerOverflow/Modules/sec_service/target/debug/incremental -L dependency=/Users/kamalmukiri/Documents/GitHub/Courses/APTRUSTSESS01/Classes/Kamal/Sep30_Modules_IntegerOverflow/Modules/sec_service/target/debug/deps
Compiling sec_service v0.1.0 (/Users/kamalmukiri/Documents/GitHub/Courses/APTRUSTSESS01/Classes/Kamal/Sep30_Modules_IntegerOverflow/Modules/sec_service)
running rustc --crate-name sec_service --edition=2021 src/main.rs --error-format=json --json=diagnostic-rendered-ansi,artifacts,future-incompat --diagnostic-width=248 --crate-type bin --emit=dep-info,link -C embed-bitcode=no -C debuginfo=2 -C split-debuginfo=unpacked -C metadata=2ad8da3b13476cdb -C extra-filename=2ad8da3b13476cdb --out-dir /Users/kamalmukiri/Documents/GitHub/Courses/APTRUSTSESS01/Classes/Kamal/Sep30_Modules_IntegerOverflow/Modules/sec_service/target/debug/incremental -L dependency=/Users/kamalmukiri/Documents/GitHub/Courses/APTRUSTSESS01/Classes/Kamal/Sep30_Modules_IntegerOverflow/Modules/sec_service/target/debug/deps --extern auth_service=/Users/kamalmukiri/Documents/GitHub/Courses/APTRUSTSESS01/Classes/Kamal/Sep30_Modules_IntegerOverflow/Modules/sec_service/target/debug/deps/libauth_service-e04437df63bd6c9b.rlib
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.95s
```


Binary and Libraries are to be part of same project:

```
$ cargo new veh-stack --bin
```

```
$ cd veh-stack
```

```
$ cargo new sensors --lib
```

```
$ cargo new control --lib
```

```
// Cargo.toml (root workspace)
```

```
[workspace]
```

```
members = ["sensors", "control", "veh-stack"]
```

Cont...

Step 5: Run the application

```
● > cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.06s
    Running `target/debug/sec_service`
Hello, world!
Going to to authenticate user: Kamal and pass: @123
Result = Ok(AuthSuccess)
○ > █
```



Introduction:

In Rust, **crates** are used to share reusable code across projects. Here are some examples of using **third-party crates** in Rust.

Modules

- Introduction
- Defining and accessing
- Visibility and Privacy
- Organizing

Crates

- Importing Third Party Crates
- Examples

"use" keyword:

What is the use of keyword "use" ?

- The use keyword is used to **bring a module, function, type, or constant into scope**.
- It makes it easier to reference items defined in other modules without needing to use the fully qualified path each time.

When to use "use"?

- Use use when you want to **access** functions, structs, or other items from a module.
- This allows you to use shorter paths to refer to those items in your code.

Modules

- Introduction
- Defining and accessing
- Visibility and Privacy
- Organizing

Crates

- Importing Third Party Crates
- Examples

Steps to use third party crates:

- 1) Find crate in crates.io
- 2) Include crate name and version in Cargo.toml
- 3) Import crate in source code
- 4) Call API/Interface/etc...

Modules

- Introduction
- Defining and accessing
- Visibility and Privacy
- Organizing

Crates

- Importing Third Party Crates
- Examples

Finding crate in website:

The screenshot shows the crates.io website with a search bar containing 'rand'. The search results are displayed for the 'rand' crate, version 0.8.5. The interface includes a sidebar with navigation links, a search bar, and a results section with details about the crate, including its description, download statistics, and links to its homepage, documentation, and repository.

crates.io

rand

Search Results for 'rand'

Displaying 1-10 of 1514 total results

Sort by Relevance

rand v0.8.5

Random number generators and other randomness functionality.

All-Time: 376,388,230
Recent: 39,471,738
Updated: 2 months ago

[Homepage](#) [Documentation](#) [Repository](#)

Include create name in project file: Method 1

Oct2_Crates > rand > rand_example > ⚙ Cargo.toml

```
1 [package]
2 name = "rand_example"
3 version = "0.1.0"
4 edition = "2021"
5
6 [dependencies]
7 rand = "*"
```

“*” means a recent version

Modules

- Introduction
- Defining and accessing
- Visibility and Privacy
- Organizing

Crates

- Importing Third Party Crates
- Examples

Include create name in project file: Method 2

```
> cargo add rand
```

Note: The above command adds the crate name in Cargo.toml as dependencies along with recent version.

Modules

- Introduction
- Defining and accessing
- Visibility and Privacy
- Organizing

Crates

- Importing Third Party Crates
- Examples

Import crate in source code and invoke API:

```
1 use rand::Rng;
2
3 fn main() {
4     let mut rng = rand::thread_rng();
5     let random_number: u32 = rng.gen_range(1..=100);
6     println!("Random number: {}", random_number);
7 }
```

Modules

- Introduction
- Defining and accessing
- Visibility and Privacy
- Organizing

Crates

- Importing Third Party Crates
- Examples

Thank you